

Fine-tuning Visual Transformers

Thomas Prior

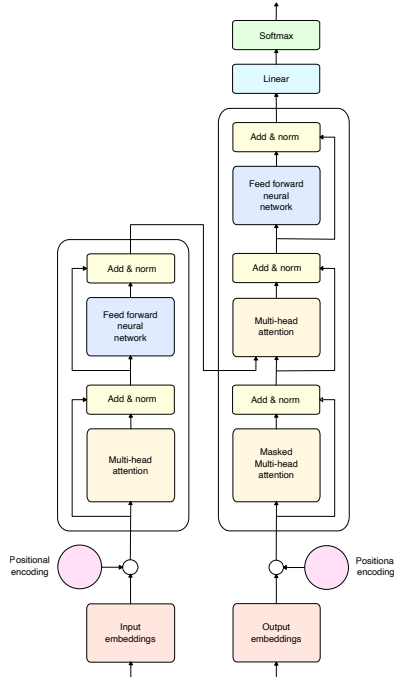


Figure 1: The original Vaswani et al. Transformer[VSP⁺17]. Note that the left-side part is the encoder, and the right-side part is the decoder, the original research featuring both.

Abstract This work explores the Transformer architecture from an encoder-only perspective, which is in contrast to the popularity of the decoder-only variant (as found in LLMs). An encoder-only Transformer is able to take vector embeddings of a variety of data formats as input, find how pieces within the input relate to other pieces within it, and output a representation of this, which can then be mapped to a fix set of categories (for example). Visual Transformers are generally decoder-only. There are a wide variety of applications to visual Transformers, from robotics, to healthcare, to natural disaster mitigation.

Experiments

ML classification The overall fine-tuning process is equivalent to the standard classification training process, as follows:

1. Taking an image dataset, each with an associated classification (supervised learning), and splitting the dataset into training, validation and testing datasets

2. Running the training dataset through the model, quantifying the output in terms of a loss function (cross-entropy loss typical during classification), iteratively adjusting the parameters of the model to encourage the output, $\hat{y}$, to be equivalent to the expected output, y , whilst simultaneously checking the efficacy of the changes using the validation set
3. Testing the model with the test set, to calculate an F1 score.

In the context of using a pretrained model for which a new dataset has associated categories with those in the training dataset used to train the model, it is also useful to calculate the F1 score *before* training.

Model architecture A visual transformer (ViT) was utilised in all the upcoming experiments. The overview of how it works is as follows.

Before inputting data into a Transformer encoder, the data is typically split into smaller portions. These portions are then represented by vector embeddings, made up of metadata of the portions, whereby a one-to-one mapping typically occurs between the portion and the vector embedding. The embedding is then further encoded with positional data, the position being in terms of the rest of the input, such as via sinusoidal functions (as in the original Transformer). The vector embeddings with positional data are then ready to be input into the Transformer. An attention mechanism, which features matrices (K, V) which extract data from the vector embeddings, and then via a dot product finds similarities between them, is able to find relations between the vector embeddings which ultimately affect the output of the Transformer, and this is done during training by iterating the values within the matrices (K, V, Q) during optimising of the loss function. Normalisation layers are used between components of the Transformer encoder to prevent the exploding/vanishing gradient problem, including after the the attention mechanism. The vector embeddings are then input into a feed-forward neural network, whereby the hidden layers typically feature 2x the quantity of neurons, in comparisong to the dimensionality of the vector embeddings. The output of the feed-forward neural network is then run through a normalisation, and output from the encoder. The Transformer encoder is typically stacked among a series of Transformer encoders, such that the aforemen-

tioned process repeats multiple times, but with different trained parameter sets within each encoder. The output of the final encoder will then be run through a linear layer, for which the dimensionality of the output is equivalent to the quantity of categories of the training dataset. The linear layer output is then run through a softmax function to determine the dominant classification. [VSP⁺17]

Methodology Hyperparameters: The model features pretrained parameters, based on ImageNet 1000, for which 13 of the iCub categories already exist, by name, in the original ImageNet training dataset (book, cell-phone, mouse, pencil case, perfume, sunglasses, wallet, beer glass, coffee mug, remote, soap dispenser, lotion, soda bottle).

Network topology: Transformer encoder-only architecture, Google’s vit-base-patch16-224, to investigate the efficacy of token-based attention mechanisms.

Dataset: iCubWorld Transformations dataset. The original dataset was made available as 256x256 dimension images, but the model architecture is bundled with an image processor. All images were extracted from a complicated file tree into one directory, and a metadata file built, for it to be labeled correctly. Preprocessing of the dataset was time-consuming.

Dataset The dataset used explicitly in upcoming experiments is the iCubWorld dataset, Part 1 only, which features 5 categories of classified images. Furthermore, implicitly the ImageNet dataset was also used, as the Transformer was pretrained using the ImageNet 1000 dataset. The mapping of the iCub World dataset to ImageNet categories was setup as follows:

```
{'book': 0, 'cellphone': 1, 'computer mouse': 2, 'pencil case': 3, 'ring binder': 4} {0: 921, 1: 487, 2: 673, 3: 709}
```

Note that, in these experiments, the iCubWorld label ID numbers are dynamically generated during training, and could change, for example if loaded in reverse order. The full list of ImageNet categories are available online.

The `with_transform()` function of the `transformers` library was used to remap the classification IDs to the ImageNet categories, on the fly, before running them through the model, both during training and testing. The image processor associated with the particular ViT model was also used. Concluding, the iCubWorld dataset was chosen due to the relevance of seeing the world from the perspective of a modern robot, such as

in terms of image quality (i.e. due to battery limitations), and dimensionality (i.e. height, field-of-view).

Code The programming by which the upcoming experiments were run can be found at the following location:

DOI [10.5281/zenodo.19189279](https://doi.org/10.5281/zenodo.19189279)

Experiment #1 This experiment simply involved taking the iCub World dataset and finding equivalent categories that the ImageNet dataset that the model was trained on (e.g. “mouse” and “computer mouse”) and running the iCub World images through the pretrained model, without any fine-tuning, and testing how well the pretrained model was at adapting to a new dataset.

Dataset 32000 samples from Part 1 of the iCub World dataset were run through the model, each of an iCub World category for which a corresponding category existed in ImageNet 1000 category dataset (4 corresponding categories), as depicted above. The original iCub World dataset was split between training and test set via a deterministic random seed.

Results

Label	ID	F1 Score
Cellphone	487	0.28
Computer mouse	673	0.48
Pencil case	709	0.10
Book jacket	921	0.25

Table 1: F1 scores when classifying test data consisting of 4 classes, via pretrained model, before fine-tuning.

The weighted F1 score was then reported as: 0.28. The above results show that the pretrained model appears to show some potential in classifying images from a new dataset, particularly in the case of the class #673, but classes #487 and #921 also hold potential.

Experiment #2 The obvious progression from the previous experiment was to take the data, fine-tune the model on the iCub World dataset, and then re-run the experiment. The hyperparameters being the quantities of training epochs and data used for training.

Dataset Equivalently to Experiment #1, the dataset only featured data from Part 1 of the iCub World dataset, for which equivalent categories (by name) existed also in the ImageNet 1000 dataset.

Fine-tuning hyperparameters It was found that a train:test subset of 128000:3200 resulted in a training time of 195.4s for 1 epoch with a batch size of 256. Therefore, to limit training time to 25 minutes, 8 training epochs were run. In terms of HuggingFace’s `transformers` library, which was used for training and testing, the following consisted of the training arguments.

```
training_args_ex.2 = TrainingArguments(
    num_train_epochs = 8,
    per_device_train_batch_size = 256,
    gradient_accumulation_steps = 8,
    gradient_checkpointing = True,
    learning_rate = 2e - 5,
    logging_steps = 10,
    eval_strategy = "epoch",
    save_strategy = "epoch",
    load_best_model_at_end = True,
    remove_unused_columns = False,
)
```

Methodology Training, validation, and test set were split via deterministic random seed in an 80:10:10 ratio. Testing was based on batch sizes of 256.

Observations Curiously, whilst testing the model, an error message cropped up regarding PyTorch calling `F.conv2d()`, which implies a convolutional neural network is used by the model, unexpectedly.

Initially, the model broke, and this was due to the iCub World numerical classes not having been mapped to the ImageNet dataset classes before the training data was used.

Results

Label	ID	Pre F1	Post F1
Cellphone	487	0.28	0.53
Computer mouse	673	0.48	0.66
Pencil case	709	0.10	0.17
Book jacket	921	0.25	0.54

Table 2: F1 scores when classifying different images via pretrained model, before and after fine-tuning.

The weighted F1 score was then reported as: 0.28. Fine-tuning appeared to increase the F1 score for all categories. This means the fine-tuned model is better at classifying the data from the iCub World dataset

into the corresponding ImageNet categories. Particularly large gains were seen with computer mice and book jackets, and seemingly the shape and design of these items is significantly more unique than other items.

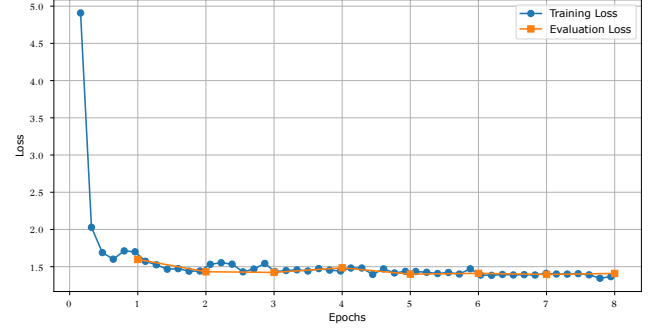


Figure 2: Line charts (zoom-in friendly; vector) to show how training and evaluation loss change per training epoch.

The trajectory of the loss function output implies logarithmic decay, meaning most of the benefits of fine-tuning are to be made during the initial training epochs, with improvement in F1 score likely to be significantly more marginal as quantity of training epochs increases further.

Experiment #3 To investigate the efficacy of more targeted fine-tuning, it seemed of interest to only focus on one class. Given how the attention mechanism works, it seemed that the low hanging fruit was class #673, with an F1-score of approximately 0.663 before fine-tuning.

Dataset The dataset only featured data of one class from Part 1 of the iCub World dataset, for which an equivalent category (by name) existed also in the ImageNet 1000 dataset, and this was ImageNet category #673 (images of a computer mouse).

Fine-tuning hyperparameters To focus more on the "fine" in fine-tuning, the hyperparameters related to batch size during training and validation were significantly lowered. These ideas were inspired by examples seen online. The training arguments were then as follows:

```

training_args_ex.3 = TrainingArguments(
    num_train_epochs = 8,
    per_device_train_batch_size = 16,
    per_device_eval_batch_size = 4,
    gradient_accumulation_steps = 32,
    gradient_checkpointing = True,
    learning_rate = 2e-5,
    logging_steps = 10,
    eval_strategy = "epoch",
    save_strategy = "epoch",
    load_best_model_at_end = True,
    remove_unused_columns = False,
)

```

Results

Epochs	F1
0	0.48
8	0.74

Table 3: F1 scores when classifying test data consisting of 1 class, #673, via pretrained model, before and after fine-tuning for 8 epochs.

Epochs	Recall
0	0.32
8	0.59

Table 4: Recall metrics when classifying test data consisting of 1 class, #673, via pretrained model, before and after fine-tuning for 8 epochs.

Epochs	Precision
0	0.99
8	1.00

Table 5: Accuracy metrics when classifying test data consisting of 1 class, #673, via pretrained model, before and after fine-tuning for 8 epochs.

Fortunately this targeted training does cause a quite significant increase in F1 score, despite only training for 8 epochs (approximately 20 minutes). This means it is making predictions regarding the iCub World dataset with a higher accuracy and precision; less false positives (samples marked as class #673 that aren't) and less false

negatives (samples not marked as class #673 that are). Specifically, the significant improvement in recall score indicates significantly less false negatives.

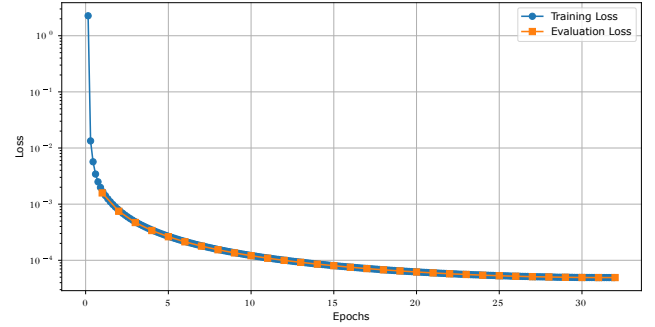


Figure 3: Line charts (zoom-in friendly; vector) to show how training and evaluation loss change per training epoch, over 32 epochs.

In the case that training epochs are significantly increased, such as to 32 as featured in Figure 3, it appears that the potential gains in F1 score rapidly drop off as epochs increase.

Compared to Experiment #2, results suggest that initial gains may be seen early with a given set of hyperparameters, if there are gains to be had by training, to begin with.

Further potential experiments The following ideas came about after previous experiments:

- Investigate the efficacy of fine-tuning a poorly classified item, class #709, with an F1-score of approximately 0.0305 before fine-tuning, was to be targeted for further training
- Investigate an obvious mismatch of a pair of categories between the iCub World dataset and the ImageNet categories, and observing if there is any significant difference to the #709 class situation
- Investigate the efficacy of identifying original ImageNet dataset images after fine-tuning

References

- [VSP⁺17] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, pages 5998–6008, 2017.